MONOTASK

Project Report

Version 1.0

PHP Short-Term Course

Programme:

BACHELOR OF COMPUTER APPLICATION (BCA)

PROJECT GUIDE:

Mr. Ravi Rai

SUBMITTED BY:

Siddhansh Jain (TCA2468573)

November, 2026

COLLEGE OF COMPUTING SCIENCES AND IT

TEERTHANKER MAHAVEER UNIVERSITY, MORADABAD

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Mr. Ravi Rai**, for his valuable guidance, constant encouragement and helpful suggestions throughout the development of this project. His feedback helped me to improve both the design and the quality of MonoTask.

I am thankful to the College of Computing Sciences and IT, Teerthanker Mahaveer University, Moradabad, for providing the facilities and the learning environment for the PHP Short-Term Course, and to all the faculty members for their support.

I also thank my family and friends for their continuous support and motivation.

Finally, I acknowledge the open-source communities behind PHP, MySQL, XAMPP and SortableJS, whose free tools and documentation made this project possible.

Siddhansh Jain
(TCA2468573)

Place: Moradabad
Date: _____

DECLARATION

I hereby declare that this Project Report titled **MonoTask – A Web-Based Integrated Productivity Workspace**, submitted by me and approved by my project guide, the College of Computing Sciences and Information Technology (CCSIT), Teerthanker Mahaveer University, Moradabad, is a bonafide work undertaken by me and it is not submitted to any other University or Institution for the award of any diploma / certificate or published any time before.

Project Name:	MonoTask (individual project)	
Student Name:	Siddhansh Jain (TCA2468573)	Signature: _____
Project Guide:	Mr. Ravi Rai	Signature: _____

Brief About the Institution

(Academic project – no external company or client was involved)

Teerthanker Mahaveer University (TMU) is a private university located in Moradabad, Uttar Pradesh, India. It offers undergraduate, postgraduate and doctoral programmes in many fields, including computing, engineering, management, pharmacy and medical sciences.

The College of Computing Sciences and Information Technology (CCSIT) of the university conducts computer-related programmes such as BCA, MCA and B.Tech (CSE). The college provides computer laboratories and faculty guidance for practical, project-based learning.

MonoTask was developed as the project work of the PHP Short-Term Course of the Bachelor of Computer Application (BCA) programme, under the guidance of Mr. Ravi Rai. The project was carried out for academic purposes and uses only free and open-source software.

Table of Contents

1 PROJECT TITLE....6
2 PROBLEM STATEMENT....6
3 PROJECT DESCRIPTION....7
3.1 Scope of the Work....7
3.2 Project Modules....8
3.3 Context Diagram (High Level)....13
4 IMPLEMENTATION METHODOLOGY....14
4.1 Development Approach....14
4.2 System Architecture....14
4.3 Development Phases....15
4.4 Security Measures....15
4.5 Testing....16
5 TECHNOLOGIES USED....17
5.1 Software Platform....17
5.2 Hardware Platform....17
5.3 Tools....18
6 ADVANTAGES OF THIS PROJECT....18
7 ASSUMPTIONS AND LIMITATIONS....18
8 FUTURE SCOPE AND FURTHER ENHANCEMENT OF THE PROJECT....19
9 PROJECT REPOSITORY LOCATION....20
10 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS....20
11 PROJECT DEMONSTRATION GUIDE....21
12 LEARNING OUTCOMES....21
13 CONCLUSION....22
14 REFERENCES....22

Appendix

A: Data Flow Diagram (DFD)

B: Entity Relationship Diagram (ERD)

C: Use Case Diagram (UCD)

D: Data Dictionary (DD)

E: Screen Shots

1 Project Title

MonoTask – a web-based integrated productivity workspace that combines a Todo List, an Eisenhower Matrix, a Pomodoro Focus Timer, a Brain Dump (Thought Pits) canvas and Quick Notes in a single application.

Item	Details
Domain	Productivity and Task Management
Project type	Individual academic project (originally planned as a group project; completed by one student)
Course / Programme	PHP Short-Term Course, Bachelor of Computer Application (BCA)
Platform	Web application (PHP 8, MySQL 8, JavaScript), runs on a standard Apache / XAMPP stack
Version	1.0

2 Problem Statement

Students and working professionals handle their daily responsibilities through a scattered mix of paper notes, phone reminders, messaging apps, calendars, separate timer applications and separate note-taking tools. This fragmentation causes missed deadlines, poor prioritisation, loss of focus and ideas that are never turned into action.

Ordinary checklist applications record **what** has to be done, but they do not help the user decide **what to do first**, do not support structured, distraction-free work sessions, and do not offer a place to think through half-formed ideas before they become tasks. As a result users keep switching between four or five tools, and important but non-urgent work is repeatedly postponed.

The specific problems addressed by this project are:

- No clear way to separate urgent work from important work (prioritisation).
- No built-in method for focused work and breaks (time management).
- Repetitive chores (daily, weekly, monthly) must be re-created manually every time.
- Large tasks cannot be broken into smaller steps with visible progress.
- Ideas, notes and loosely connected thoughts are kept in different places from the tasks they belong to.
- Many existing tools need paid subscriptions, internet accounts or heavy installations.

MonoTask solves these problems by offering one lightweight, private, multi-user web workspace in which every user has separate data for tasks, priorities, focus sessions, idea maps and notes, using only open-source software that runs on an ordinary XAMPP installation.

3 Project Description

MonoTask is a multi-user web application written in PHP 8 with a MySQL 8 database (accessed through PDO) and plain JavaScript on the browser side. A visitor registers an account, logs in, and is then taken to the Todo module. A navigation bar on every page leads to the other modules. Every record created in any module is stored against the owner's username, and every database query filters on the logged-in username, so a user can only see and change his or her own data. The interface uses one consistent dark theme with subtle hover effects.

The application contains five functional modules supported by a common access and navigation layer:

#	Module	Purpose	Main database tables
1	Todo List Management	Prioritised task list with due dates, recurrence, subtasks, pinning, drag-and-drop ordering, search and trash	tasks, subtasks
2	Eisenhower Matrix	Four colour-coded quadrants that classify tasks by urgency and importance	eisenhower
3	Pomodoro Focus Timer	Focus / break timer with progress ring, session statistics and notifications	None (browser localStorage for colours)
4	Brain Dump (Thought Pits)	Visual canvas of draggable idea nodes that can be linked with arrows	thought_pits, pit_nodes, pit_connections
5	Quick Notes	Rich-text note editor with images, tables, checklists, search and export	quick_notes

3.1 Scope of the Work

Inside the scope of this version (v1.0)

- User registration, login, logout and session-protected pages with per-user data isolation.
- Todo List with priorities, due dates, recurring tasks, subtasks with progress bar, pinning, live search, sorting, drag-and-drop ordering, soft delete, restore and permanent delete.
- Eisenhower Matrix with four quadrants, completion toggle, soft delete, restore and permanent delete.
- Pomodoro timer with focus, short-break and long-break modes, statistics, notifications and colour customisation.
- Brain Dump canvas with multiple thought pits, nodes, connections, zoom, pan, snapping and automatic tidy-up.
- Quick Notes rich-text editor with auto-save, search, image/table/checklist support and export.
- Responsive dark user interface that works on desktop and smaller screens.

Outside the scope of this version (planned for future work)

- Productivity dashboard and statistical analysis of completed tasks.
- Team collaboration, shared projects and task assignment to other users.
- Custom colour labels / tags on tasks, user profile page with profile picture and settings page.
- Server-side logging of Pomodoro sessions and linking sessions to tasks.
- E-mail or push reminders, password reset, e-mail verification and calendar integration.
- Native mobile applications and AI-based suggestions.

Comparison with the project synopsis

The synopsis (version 1.0, September 2026) described the planned system. During development the project grew in some areas and some planned items were postponed. The table summarises the differences so that the report matches the delivered software.

Item	Planned in synopsis	Status in delivered system
Authentication	Registration and login with PHP sessions	Delivered (passwords hashed with bcrypt)
Todo List	Create, edit, delete, filter, pin, priority sorting	Delivered, plus recurrence, subtasks, drag-and-drop, search, trash
Eisenhower Matrix	Four quadrants	Delivered, plus completion toggle and trash
Pomodoro Timer	Timer that logs sessions to the server	Timer delivered; session statistics kept in the browser only; server logging postponed
Dashboard and insights	Aggregated statistics and charts	Not yet implemented (future scope)
Task categories	Customisable colour-coded categories	Not yet implemented (future scope)
Brain Dump, Quick Notes	Not in synopsis	Added as additional modules

3.2 Project Modules

The functions of the system are described below. As required by the template, registration and login are treated as a supporting feature and not as a module.

Main Menu and Navigation (common to all pages)

After login a navigation bar (`includes/nav.php`) is shown at the top of every module page. It contains the application name, a link to each module, the name of the logged-in user and a Logout button. The link of the page being viewed is highlighted. Pages are opened through these links; there is no separate menu page.

- **Access control:** every page begins with `session_start()` and redirects to `login.php` when no user is logged in.
- **Logout:** `logout.php` clears the session data, removes the session cookie and destroys the session.
- **Registration** (`index.php`): username, password, date of birth and e-mail are all mandatory; a username that already exists is rejected; the password is stored only as a bcrypt hash created by `password_hash()`.
- **Login** (`login.php`): the stored hash is checked with `password_verify()`; a wrong username or password gives one generic error message; an already logged-in user is sent straight to the Todo page.

Module 1 – Todo List Management

Lets a user capture work as tasks, rank them by priority, attach deadlines, repeat them automatically and break them into smaller steps. Main page: `todo.php`.

Function	Description
Add task	Title plus priority (Low, Medium, High, Critical – default Medium) and an optional due date and time. The task is saved as pending at the end of the user's ordering.
Recurring tasks	Optional recurrence: daily, weekly (chosen weekday), monthly (chosen date 1–28) or custom (every 1–365 days) at a chosen time. A live preview sentence such as "Every Monday at 9:00 AM" is shown while filling the form. When the Todo page is opened, every recurring task whose next-due time has passed is reset to pending and its next-due time is recalculated.
Automatic ordering	Pinned tasks first, then by priority (Critical → Low), then by the user's own drag-and-drop order.
Search and sort	A search box filters tasks by title as the user types; a sort list orders them by default order, priority, due date or A→Z. Both work in the browser without reloading the page.
Drag-and-drop ordering	A drag handle on each task lets the user reorder the list; the new order is saved immediately in the background through `api/reorder.php`.
Task card	Each task shows priority, due date (overdue tasks are highlighted in red), a recurring marker, a subtask counter and a progress bar. Clicking the card expands it.
Subtasks and progress	Inside the expanded card subtasks can be added, ticked and deleted. These actions are sent with `fetch` and only the affected card is replaced, so the card stays open and the progress bar updates without a page reload.
Pin, priority, due date	From the expanded card a task can be pinned or unpinned, its priority changed and its due date set or changed.
Trash (soft delete)	Deleting a task only stores the deletion time. Deleted tasks appear in a collapsible Trash section where they can be restored or removed permanently (their subtasks are deleted with them).
Completed section	Tasks whose status is "completed" are listed in a separate collapsible section.

Implementation	Details
Source files	`todo.php`, `includes/task_card.php`, `modules/tasks/` (add, delete, restore, permanent_delete, pin, priority, due_date, recurring), `modules/subtasks/` (add, toggle, delete), `api/reorder.php`, `assets/js/todo.js`, `assets/js/ajax-subtasks.js`, `assets/css/todo.css`, `assets/css/theme.css`
Database tables	`tasks`, `subtasks`
Techniques	Post-Redirect-Get after every form action; PDO prepared statements; every update and delete also checks `username`; SortableJS for drag-and-drop; `fetch` for background updates; foreign key with ON DELETE CASCADE between subtasks and tasks

Module 2 – Eisenhower Matrix

Helps the user decide what to do first by placing every task in one of four quadrants according to urgency and importance. Page: `eisenhower.php`.

Quadrant	Colour	Meaning	Stored value (quadrant column)
Important and Urgent	Red	Do First	imp_urg
Important and Not Urgent	Blue	Schedule	imp_nurg
Not Important and Urgent	Amber	Delegate	nimp_urg
Not Important and Not Urgent	Grey	Eliminate	nimp_nurg

- A task is added directly into a quadrant using that quadrant's own input box.
- **Done / Undo** button toggles a task between pending and completed; completed tasks are shown struck-through and faded.
- Deleting a task is a soft delete. A collapsible Trash section lists deleted tasks together with their quadrant and offers Restore and permanent Remove.
- The four quadrants are shown as a 2 × 2 grid on large screens and as a single column on screens narrower than 600 px.
- All queries and changes are restricted to the logged-in user (`username` condition).

Source file: `eisenhower.php` (page logic, markup and styles in one file). Database table: `eisenhower`.

Module 3 – Pomodoro Focus Timer

Supports focused work using the Pomodoro Technique. The module (`pomodoro.php`) runs entirely in the browser with JavaScript; it needs the database only for the login check.

- **Three modes:** Pomodoro (focus) 25 minutes, Short Break 5 minutes, Long Break 15 minutes, selectable with tabs.
- **Progress ring:** an SVG circle shows the remaining time and the large digital countdown is shown in its centre.
- **Controls:** Start / Pause, Reset and Skip.
- **Automatic cycle:** when a focus session ends the timer moves to a break – a long break after every fourth focus session, otherwise a short one – and returns to focus after the break. Four dots show progress through the cycle.
- **Statistics:** counters for completed sessions, focus minutes and breaks (kept while the page is open).
- **Motivational messages:** a random message suited to the current mode.
- **Notifications:** the browser asks for permission and shows a notification when a session ends.
- **Colour customisation:** a separate colour can be chosen for each mode, is saved in the browser's `localStorage`, and can be restored to the defaults with one button.

Module 4 – Brain Dump (Thought Pits)

A free-form visual space for ideas. A user creates one **thought pit** for each topic or scenario, and inside it places **nodes** (small note cards) that can be moved freely and joined with arrows. Files: `brain_dump.php`, `brain_dump.js`, `brain_dump_api.php`, `brain_dump_style.css`.

Function	Description
Thought pit list	Grid of the user's pits showing name, accent colour and node count; a pit can be created (with a chosen accent colour), opened or deleted.
Add and edit nodes	A node has a heading (required, up to 120 characters), optional details (up to 1000 characters) and a border colour. Details appear when the pointer is over the node; double-clicking opens an edit window with a Delete option.
Drag and snap	Nodes can be dragged anywhere; guide lines and snapping help align a node with its neighbours. New positions are saved to the database.
Connections	Hovering over a node shows "+" ports on its four sides; dragging from one port to another node draws an arrow. A connection is deleted with the cross shown when the pointer is over it. Duplicate links and links from a node to itself are rejected.
Zoom and pan	Zoom buttons, Ctrl + mouse wheel or touchpad pinch zoom at the pointer; the wheel or two-finger scroll pans; the canvas can also be dragged. "Fit" shows all nodes.
Housekeeping	"Tidy up" arranges nodes automatically, "Reset connections" removes all links and "Delete all" empties the pit after a confirmation window. A status bar shows the number of nodes and connections.

Implementation	Details
Back end	`brain_dump_api.php` is a JSON API with twelve actions: list_pits, create_pit, delete_pit, get_pit, create_node, update_node, update_positions, delete_node, create_connection, delete_connection, reset_connections, clear_pit. Read actions accept GET; all changes require POST. Colours are validated against a hex pattern, sides against a fixed list, and every action checks that the pit or node belongs to the logged-in user. Position updates are saved in one database transaction.
Front end	Plain JavaScript with pointer events; links are drawn as SVG curves with arrow heads; the canvas uses CSS transforms for zoom and pan.
Database tables	`thought_pits`, `pit_nodes`, `pit_connections` (foreign keys with ON DELETE CASCADE; a unique key prevents duplicate links)

Module 5 – Quick Notes

A rich-text note book for longer text that does not belong in a task. Files: `quick_notes.php`, `quick_notes.js`, `quick_notes_api.php`, `quick_notes_style.css`.

- **Notes list** with search; notes are sorted by last change; the list can be shown or hidden.
- **Formatting toolbar:** font and font size, bold, italic, underline, strikethrough, text colour, highlight colour, alignment, headings H1–H3, paragraph styles, bullet and numbered lists, checklists, quotes and divider lines; undo, redo and clear formatting.
- **Insert:** links, tables (with add row, add column and delete table), images (inserted, pasted or dropped, resizable by dragging) and the current date and time.
- **Keyboard help:** Ctrl+B / I / U, Ctrl+Z / Y, Ctrl+S to save, Ctrl+/ to jump to a heading, and Markdown-style shortcuts such as "# ", "- " and "1. " at the start of a line.
- **Auto-save:** changes are saved automatically about one second after typing stops; a word and character counter is shown.
- **File menu:** save, duplicate, export as HTML or plain text, print, and delete.
- **Images** are reduced in size in the browser before saving and stored inside the note; a very large note is refused with a clear message.

Notes are stored in the `quick_notes` table (title, short preview and the note content as HTML). The API (`quick_notes_api.php`) offers list, get, create, save, duplicate and delete actions, rejects oversized content and always restricts access to the owner's notes.

Common Services Used by All Modules

- **Database connection:** one shared file (`db.php`) opens a PDO connection with exceptions enabled and UTF-8 (utf8mb4); settings can be given through environment variables for deployment. `config/db.php` simply includes it for the modular files.

- **Security measures:** prepared statements everywhere (protection against SQL injection), `htmlspecialchars()` when printing user text (protection against cross-site scripting), password hashing, session checks, ownership checks on every change, input validation and length limits, and an `.htaccess` rule that blocks direct download of `.env` and `.sql` files.
- **User interface:** shared dark theme; `theme.css` adds slight zoom on hover for cards, navigation links and buttons and respects the "reduce motion" setting of the operating system.

3.3 Context Diagram (High Level)

The context diagram shows MonoTask as a single process. The only human actor is the registered **User**. The web browser provides two services used by the Pomodoro module: notifications and local storage of the chosen colours. The database is inside the system boundary and therefore not shown as an external entity. The detailed Level-1 data flow diagram is given in Annexure A.

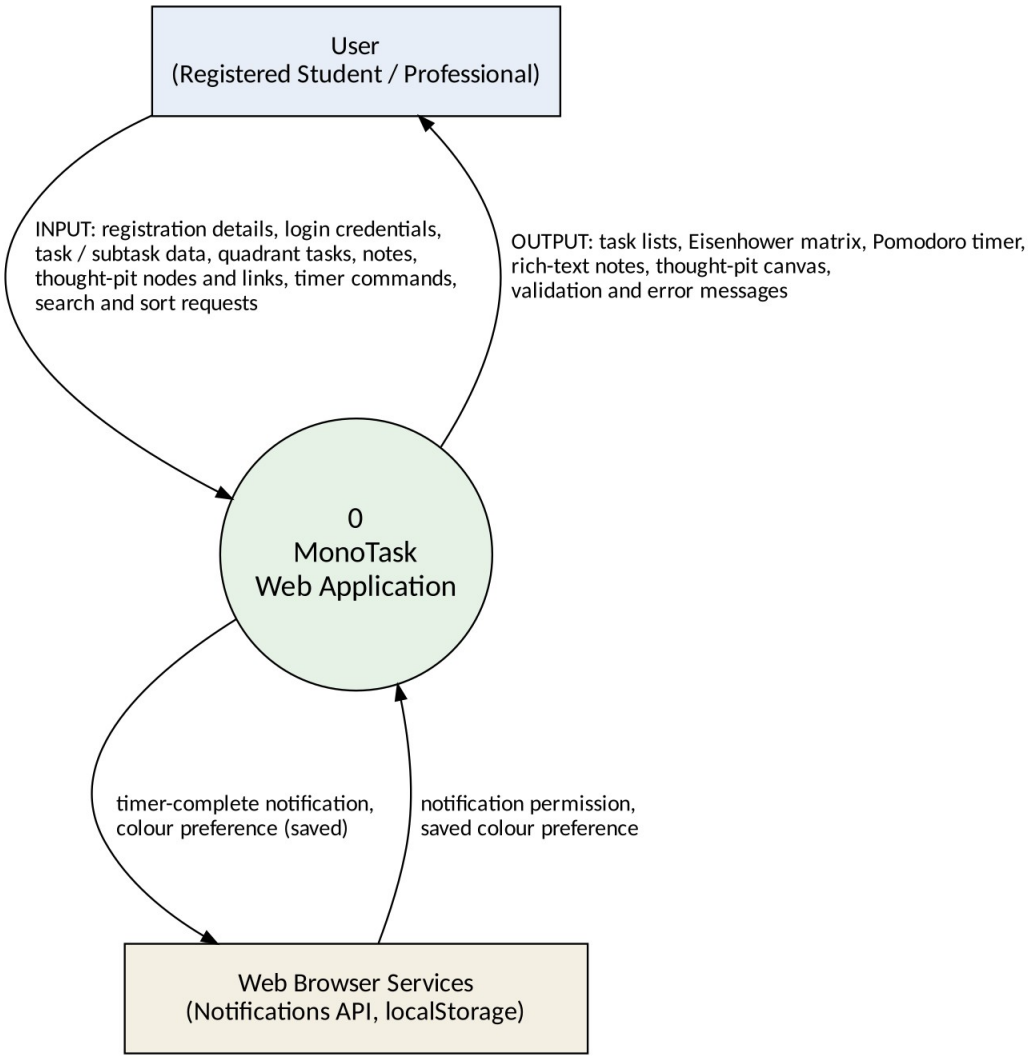


Figure 3.1 – Context diagram (Level 0)

4 Implementation Methodology

4.1 Development Approach

The project followed an iterative and incremental approach. A small working version (registration, login and a simple todo list) was built first; each following module was added in a separate cycle of design, coding and testing, and the earlier code was reorganised whenever it became hard to maintain. For example, the first single-file todo page was later split into a modular structure with separate folders for configuration, shared page parts, action handlers, background APIs and static assets.

4.2 System Architecture

MonoTask is a three-tier client–server web application:

Tier	Description
Presentation (browser)	HTML5 pages, CSS3 styles and JavaScript. Handles display, drag-and-drop, live search, canvas drawing, rich-text editing and the Pomodoro timer.
Application (Apache + PHP 8)	Page scripts and action handlers that check the session, validate input, run database queries and return HTML or JSON.
Data (MySQL 8, InnoDB)	Database `todo_list` holding eight tables with foreign keys and indexes (see Annexure B and D).

Two request patterns are used:

- **Form post with Post-Redirect-Get:** the Todo, Eisenhower, registration and login actions submit an HTML form, PHP changes the data and redirects back to the page, so refreshing the browser never repeats the action.
- **Background JSON requests (fetch):** subtask changes, task reordering, all Brain Dump actions and all Quick Notes actions are sent in the background; the server answers with JSON or an updated card and the page is updated without a reload.

4.3 Development Phases

#	Phase	Work done
1	Requirement analysis	Identified the problems of scattered productivity tools; selected the Todo List, Eisenhower Matrix and Pomodoro Technique as core ideas; wrote the synopsis.
2	Design	Designed the database tables, per-user data isolation, page flow and dark user interface.
3	Authentication	Registration, login, logout, password hashing and session protection.
4	Todo List	Tasks, priority, due date, pinning, recurrence, subtasks, drag-and-drop, search, sort and trash; modular re-organisation of the code.
5	Eisenhower Matrix and Pomodoro	Four-quadrant matrix with completion toggle and trash; timer with progress ring, statistics, notifications and colour settings.
6	Brain Dump and Quick Notes	Canvas with nodes and links and its JSON API; rich-text editor with auto-save and its API.
7	Database hardening	Added missing columns and indexes, a foreign key from subtasks to tasks, and the required database privilege.
8	Testing and documentation	Functional testing of each module, correction of defects, synopsis and this report.

4.4 Security Measures

Threat	Measure used in MonoTask
SQL injection	All queries use PDO prepared statements with bound parameters.

Threat	Measure used in MonoTask
Stored passwords	Passwords are saved only as bcrypt hashes (`password_hash`, verified with `password_verify`).
Cross-site scripting	User text is escaped with `htmlspecialchars()` before it is printed in a page.
Access to other users' data	Every select, update and delete contains `username = logged-in user`; the Brain Dump API also checks that a pit or node belongs to the user.
Unauthorised page access	Each page and API checks the session and redirects (or returns HTTP 401) when no user is logged in.
Accidental repeated actions	Post-Redirect-Get after form actions; changing APIs accept POST only.
Exposure of configuration	`.htaccess` blocks `.env` and `.sql` files; database settings can come from environment variables.
Invalid input	Colours, enumerated values and lengths are validated on the server; empty titles are rejected.

4.5 Testing

Each module was tested by using it as an ordinary user would and by trying wrong or unusual input. The main test cases are listed below with the expected result. Two test accounts were used to confirm that data is not shared between users.

#	Test case	Expected result	Module
1	Register with an existing username	Error "Username already taken!"; no new row	Authentication
2	Register with an empty field	Error "All fields are required!"	Authentication
3	Login with a wrong password	Generic error message; no session created	Authentication
4	Open `todo.php` without logging in	Redirected to the login page	Access control
5	Add a High priority task with a due date	Task appears above Medium tasks with a due-date badge	Todo
6	Add a subtask inside an open card	Card stays open; counter and progress bar update	Todo
7	Drag a task to a new position and reload	The new order is kept	Todo
8	Delete a task, then restore it	Task moves to Trash and returns to the list	Todo
9	Type in the search box	Only matching tasks stay visible while typing	Todo
10	Add a task in each quadrant and mark one Done	Tasks stay in their quadrants; the done task is struck through	Eisenhower
11	Complete four focus sessions	A long break (15 min) follows the fourth session	Pomodoro
12	Change a timer colour and reload	The chosen colour is kept	Pomodoro

#	Test case	Expected result	Module
13	Create two nodes and connect them	An arrow is drawn and kept after reload	Brain Dump
14	Try to connect a node to itself	Rejected with a message	Brain Dump
15	Type text, insert an image and wait	Note is saved automatically and reloads with the image	Quick Notes
16	Log in as a second user	Sees none of the first user's tasks, pits or notes	All

5 Technologies Used

5.1 Software Platform

a) Front-end

- **HTML5** – page structure, forms, `contenteditable` editor area.
- **CSS3** – flexbox and grid layouts, CSS variables, transitions, media queries for small screens, print style for notes.
- **JavaScript (ES6+, no framework)** – DOM handling, Fetch API, pointer events, Notification API, localStorage, FileReader and canvas (image reduction).
- **SVG** – Pomodoro progress ring and Brain Dump connection arrows.
- **SortableJS 1.15** (loaded from a CDN) – the only third-party library, used for drag-and-drop ordering of tasks.

b) Back-end

- **PHP 8.2** – server-side scripting, sessions, `password_hash` / `password_verify`.
- **PDO (MySQL driver)** – database access with prepared statements and exception mode.
- **MySQL 8.0 (InnoDB, utf8mb4)** – relational database `todo_list` with foreign keys, unique keys and indexes.
- **Apache HTTP Server** – web server provided by XAMPP, with `.htaccess` rules.

5.2 Hardware Platform

Item	Requirement
Processor	Intel / AMD, dual-core or better
RAM	4 GB minimum
Hard disk	250 GB HDD / SSD (the application itself needs well under 100 MB)
Operating system	Windows 10 / 11 (development system: Windows 11)
Browser	Current Google Chrome or Microsoft Edge (Firefox and Safari expected to work)

Item	Requirement
Network	Local server needs none; the SortableJS library is loaded from the internet

5.3 Tools

Tool	Use
XAMPP 8.2.12 (Apache, MySQL 8.0.44, PHP 8.2.12)	Local development server
phpMyAdmin 5.2.1	Creating, altering and exporting database tables
Visual Studio Code	Writing and editing source code
Google Chrome / Microsoft Edge developer tools	Debugging JavaScript, network requests and layout
Git and GitHub	Version control (as planned in the synopsis)

Total cost of software: ₹0 – every tool listed is free or open source.

6 Advantages of this Project

- **One workspace instead of five tools** – tasks, priorities, focus timer, idea maps and notes in a single login.
- **Helps decide what to do first** – priority levels, pinning and the Eisenhower quadrants separate urgent from important work.
- **Less repeated typing** – recurring tasks come back automatically on the chosen schedule.
- **Visible progress** – subtask counters and progress bars show how far a larger task has advanced.
- **Safe to delete** – the trash with restore prevents accidental data loss.
- **Private** – each user's data is separated and passwords are never stored in readable form.
- **Fast and smooth** – background requests update only the part of the page that changed.
- **Zero cost and low requirements** – runs on an ordinary computer with free software.
- **Easy to extend** – the modular folder structure and relational database allow new modules to be added without rewriting existing ones.

7 Assumptions and Limitations

Assumptions

- Users have a modern browser with JavaScript enabled.

- The web server and MySQL server are running whenever the application is used.
- A username is unique and is not changed after registration, because all data is linked through it.
- Dates and times follow the time zone of the server.
- Each user stores a moderate amount of data (a few thousand tasks and notes).

Known limitations of version 1.0

- Recurring tasks are updated when the Todo page is opened; there is no background scheduler.
- Pomodoro statistics are not saved and restart from zero when the page is reloaded.
- The Todo List and the Eisenhower Matrix use separate tables, so Eisenhower tasks have no subtasks or priorities yet.
- Forms do not yet use CSRF tokens, and there is no password-reset or e-mail verification.
- The link between a record and its owner is made through the `username` column without a database foreign key to `user_data`.
- The dashboard, labels, profile page and collaboration features planned in the synopsis are not part of this version.

8 Future Scope and Further Enhancement of the Project

Enhancement	Description
Productivity dashboard	Statistics on completed and pending tasks per quadrant, early or late completion (the `completed_at` column is already present), charts and improvement hints.
Labels and tags	Short custom colour-coded labels shown on each task in the Todo List and the Eisenhower Matrix.
Profile and settings	Profile page with optional profile picture, a settings page and a sliding side menu.
Eisenhower upgrade	Subtasks, priorities, due dates, filters and sorting for each quadrant, and customisable quadrant colours.
Team collaboration	Teammate search and requests with privacy settings, shared projects, and assignment of tasks with due dates and priorities.
Reminders	E-mail or browser reminders for approaching deadlines.
Pomodoro integration	Save sessions on the server and attach them to tasks to measure real effort.
Security improvements	CSRF tokens, login attempt limits, password reset and e-mail verification.
Deployment and access	Hosting on a free PHP host, installable web app (PWA) and later native mobile apps.
Smart features	Calendar integration and AI-assisted priority suggestions.

9 Project Repository Location

S#	Project Artifacts (softcopy)	Location	Verified by Project Guide	Verified by Lab In-Charge
1	Project Synopsis (Final Version)	File: MonoTask_Synopsis_V10.pdf	Name and Signature	Name and Signature
2	Project Report (Final Version)	File: MonoTask Project Report (this document)	Name and Signature	Name and Signature
3	Source code	Development system: `C:\xampp\htdocs\Mini_Project\`	Name and Signature	Name and Signature
4	Database (structure and sample data)	MySQL database `todo_list`; phpMyAdmin export dated 01-Nov-2026	Name and Signature	Name and Signature
5	Source code repository	GitHub repository URL: to be added by the student	Name and Signature	Name and Signature

10 Definitions, Acronyms, and Abbreviations

Abbreviation	Description
AJAX / Fetch	Sending a request to the server in the background and updating part of a page without reloading it
API	Application Programming Interface – here a PHP file that answers with JSON data
bcrypt	One-way password hashing algorithm used by `password_hash()`
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery – an attack that makes a logged-in browser send an unwanted request
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model – the browser's tree of page elements
ERD	Entity-Relationship Diagram
FK / PK / UK	Foreign Key / Primary Key / Unique Key
HTML	HyperText Markup Language
JSON	JavaScript Object Notation – text format for exchanging data
LAMP / XAMPP	Linux, Apache, MySQL, PHP stack / its Windows package (Apache, MySQL, PHP, Perl)
PDO	PHP Data Objects – database access layer of PHP
PRG	Post-Redirect-Get – redirecting after a form post so a refresh does not repeat the action
SQL	Structured Query Language

Abbreviation	Description
SVG	Scalable Vector Graphics
UCD	Use Case Diagram
XSS	Cross-Site Scripting – injecting script through unescaped text
Soft delete	Marking a record as deleted (with a time stamp) instead of removing it, so it can be restored
Thought pit / node	A Brain Dump workspace for one topic / a note card inside it
Eisenhower Matrix	Method that sorts tasks by urgency and importance into four quadrants
Pomodoro Technique	Method that alternates 25-minute focus sessions with short breaks

11 Project Demonstration Guide

The following sequence shows every module in about ten minutes.

1. **Register** a new account on the registration page, then **log in**. Show that a wrong password is refused.
2. **Todo List:** add a Critical task with a due date and a daily recurring task; show the priority order, search box and sort list.
3. Open a task card, add two subtasks, tick one and show the progress bar; pin the task and drag another task to a new position.
4. Delete a task, show it in the Trash, then restore it.
5. **Eisenhower Matrix:** add one task to each quadrant, mark one Done and delete another; show the Trash.
6. **Pomodoro:** start the timer, switch modes, change a colour and show the statistics cards.
7. **Brain Dump:** create a thought pit, add three nodes, drag them, connect them with arrows, use zoom, Fit and Tidy up.
8. **Quick Notes:** create a note, apply formatting, insert a table and an image, show auto-save and search, export as HTML.
9. **Log out**, log in as a second user and show that none of the first user's data is visible.

12 Learning Outcomes

Working on MonoTask gave practical experience in the following areas:

- **Server-side programming:** building a complete PHP application with sessions, form handling, validation and the Post-Redirect-Get pattern.

- **Database design:** designing eight related tables, choosing data types and keys, adding indexes and foreign keys with cascading delete, and altering a live schema safely; solving a MySQL privilege problem (REFERENCES) that blocked a foreign key.
- **Secure coding:** protecting against SQL injection with prepared statements, against XSS with output escaping, storing passwords with bcrypt and checking record ownership on every request.
- **Client-side programming:** DOM manipulation, Fetch API and JSON, pointer events, SVG drawing, drag-and-drop, `contenteditable` rich-text editing, `localStorage` and the Notification API.
- **User-interface design:** consistent dark theme, responsive layouts, and subtle hover effects that respect the "reduce motion" preference.
- **Software structure:** moving from single-file pages to a modular folder layout with shared connection and navigation files, and removing duplicated code.
- **Problem solving and debugging:** finding the causes of errors such as undefined sessions, duplicated database connection files, collapsing flex layouts and cards that closed after an update.
- **Documentation and planning:** writing a synopsis, drawing context, data-flow, ER and use-case diagrams, and preparing a data dictionary and test plan.

13 Conclusion

MonoTask brings task management, prioritisation, focused working, idea mapping and note taking into one secure, low-cost web application. A user can plan tasks with priorities, due dates, recurrence and subtasks, classify them with the Eisenhower Matrix, work in Pomodoro sessions, collect loosely connected ideas on a Brain Dump canvas and keep detailed notes – all behind a personal login with separated data.

The project met its main objectives: a secure authentication system, a full-featured Todo List, the Eisenhower Matrix and the Pomodoro timer were completed, and two extra modules (Brain Dump and Quick Notes) were added. The productivity dashboard, labels and collaboration features planned in the synopsis remain as future work, and the database and folder structure were prepared so that they can be added without major redesign. Beyond the software itself, the project strengthened the author's skills in PHP, MySQL, JavaScript, secure coding and structured documentation.

14 References

The following sources were used for this project:

S#	Reference Details	Owner	Version	Date
1	MonoTask Project Synopsis	Siddhansh Jain	1.0	Sep-2026
2	PHP Manual – sessions, PDO, password hashing (php.net)	The PHP Group	8.x	2026
3	MySQL 8.0 Reference Manual (dev.mysql.com)	Oracle Corporation	8.0	2026
4	MDN Web Docs – JavaScript, HTML, CSS, Fetch, SVG, Notifications (developer.mozilla.org)	Mozilla	–	2026
5	SortableJS documentation (github.com/SortableJS/Sortable)	SortableJS contributors	1.15	2026
6	OWASP Cheat Sheet Series – SQL injection, XSS and password storage (owasp.org)	OWASP Foundation	–	2026
7	The Eisenhower Matrix – task prioritisation framework	Attributed to Dwight D. Eisenhower	–	–
8	The Pomodoro Technique – time management method	Francesco Cirillo	–	–
9	XAMPP and phpMyAdmin documentation (apachefriends.org, phpmyadmin.net)	Apache Friends / phpMyAdmin team	8.2 / 5.2	2026

Annexure A
Data Flow Diagram (DFD)
(Mandatory)

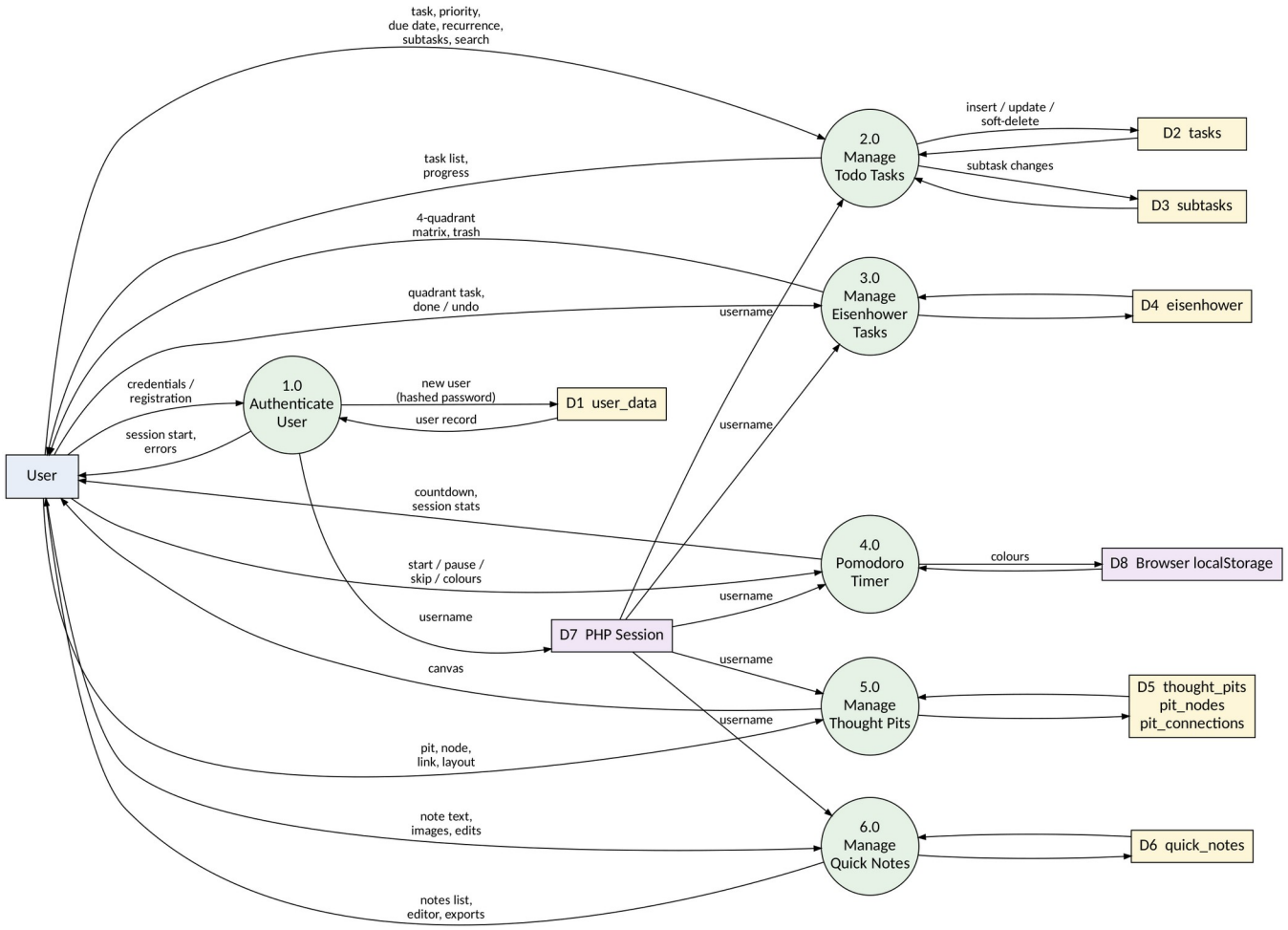


Figure A.1 – Level 1 DFD (the Level 0 context diagram is Figure 3.1)

Description of processes and data stores

ID	Name	Description
1.0	Authenticate User	Registers a new user (password stored as a hash), checks login credentials and starts or ends the PHP session.
2.0	Manage Todo Tasks	Adds, lists, searches, reorders, pins, prioritises, schedules, repeats, soft-deletes and restores tasks and manages their subtasks.
3.0	Manage Eisenhower Tasks	Adds tasks to a quadrant, toggles done / undone, soft-deletes, restores and permanently removes them.
4.0	Pomodoro Timer	Runs the focus / break countdown in the browser, shows statistics and notifications, and keeps colour choices in localStorage.
5.0	Manage Thought Pits	Creates pits, nodes and connections, saves node positions, and deletes or clears them.
6.0	Manage Quick Notes	Creates, auto-saves, searches, duplicates, exports and deletes rich-text notes.
D1	user_data	User accounts with hashed passwords.
D2 / D3	tasks / subtasks	Todo tasks and their subtasks.
D4	eisenhower	Tasks placed in the four quadrants.
D5	thought_pits, pit_nodes, pit_connections	Brain Dump pits, nodes and links.
D6	quick_notes	Rich-text notes.
D7	PHP session	Holds the logged-in username that processes 2.0 – 6.0 use to filter data.
D8	Browser localStorage	Saved Pomodoro colours.

Annexure B
Entity-Relationship Diagram (ERD)
(Mandatory)

Figure B.1 - Task domain: user_data, tasks, subtasks, eisenhower

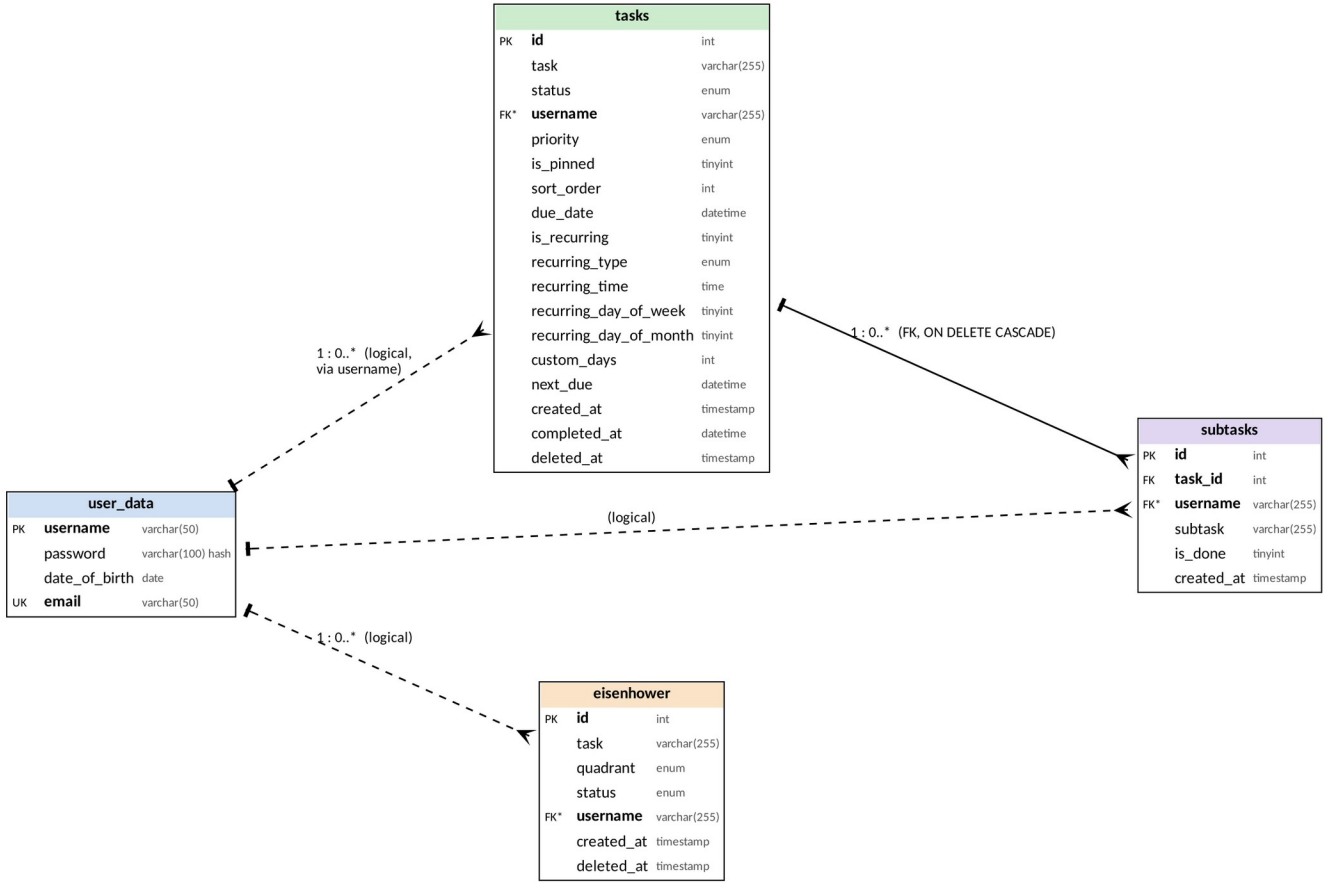
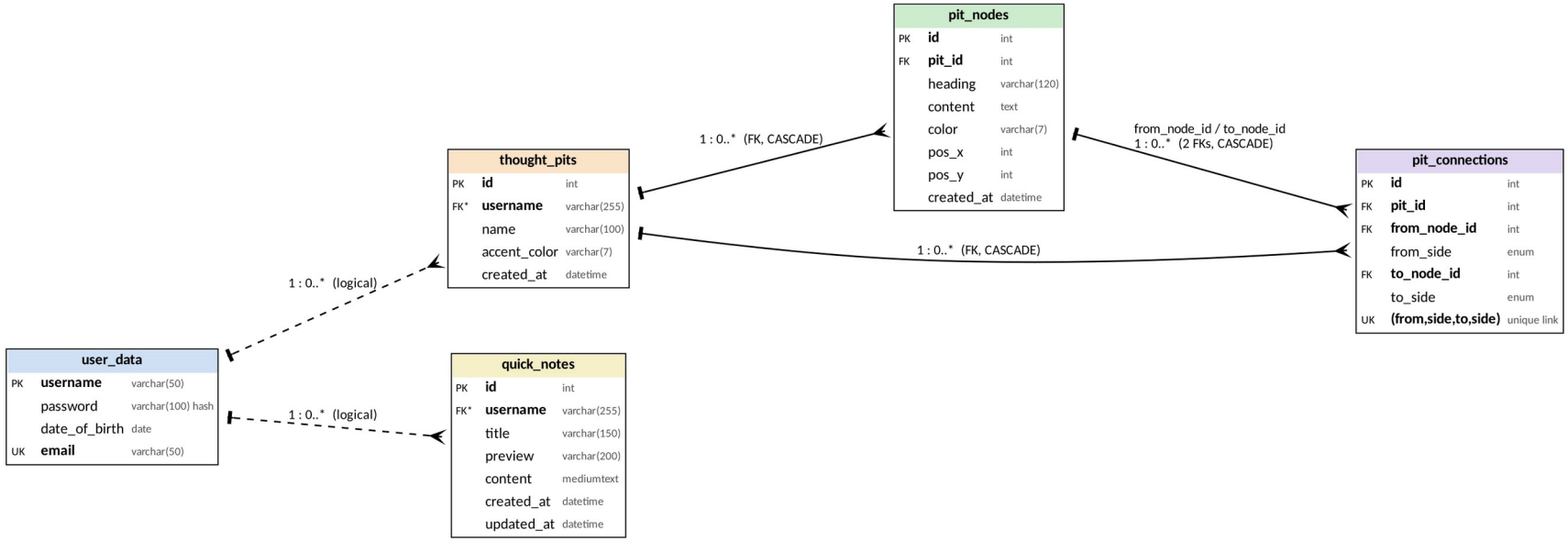


Figure B.2 - Brain Dump and Quick Notes domain



Figures B.1 and B.2 – Entity-relationship diagrams (crow's foot notation)

Symbol	Meaning
PK / UK	Primary key / unique key
FK	Foreign key enforced by the database (ON DELETE CASCADE): subtasks.task_id → tasks.id; pit_nodes.pit_id, pit_connections.pit_id → thought_pits.id; pit_connections.from_node_id and to_node_id → pit_nodes.id
FK* (dashed line)	Logical link through the `username` column to user_data.username. It is maintained by the application code (every query filters on the logged-in user) and is not a database constraint.
1 : 0..*	One record on the left can have zero or many records on the right

Annexure C
Use-Case Diagram (UCD)
(Optional)

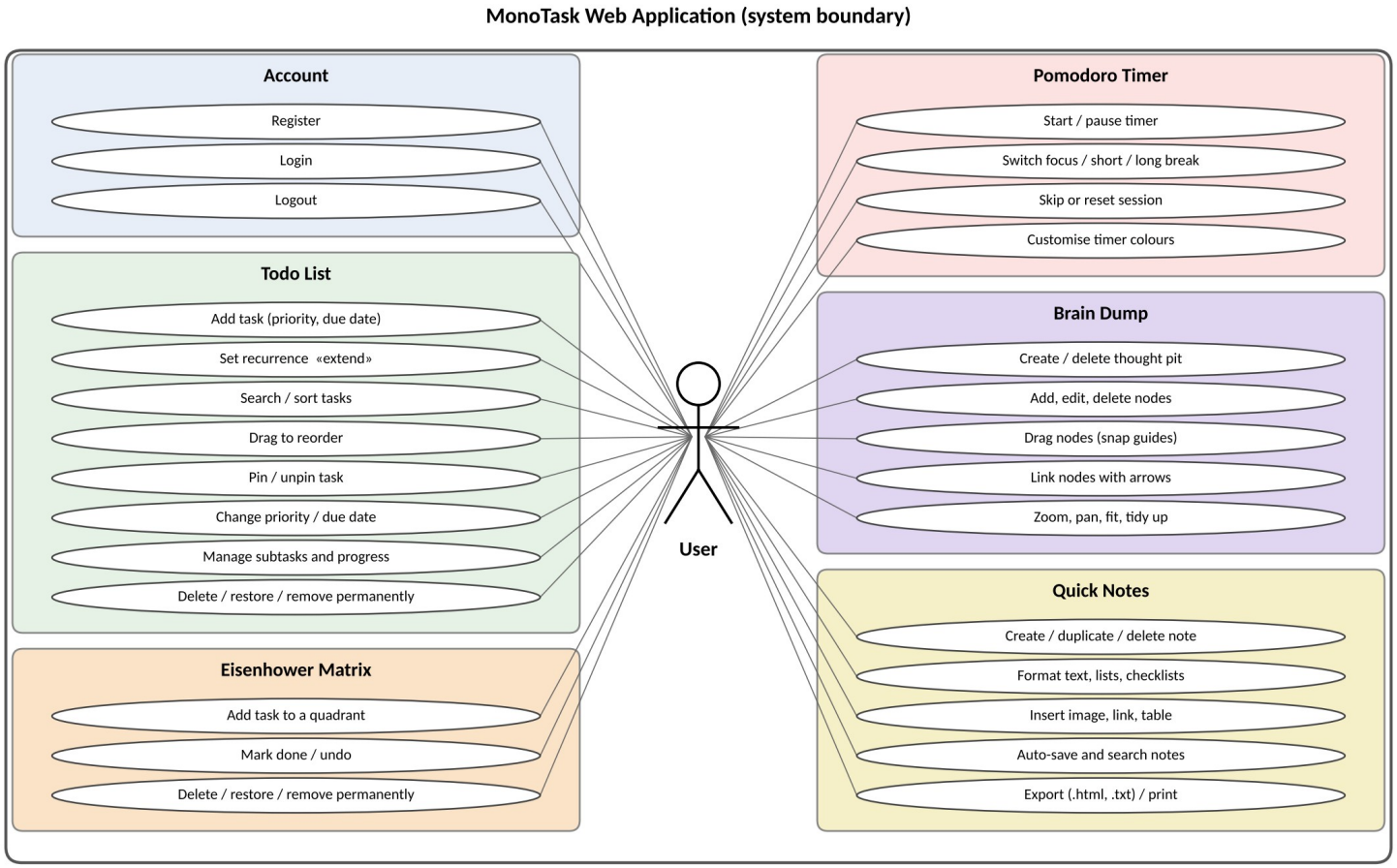


Figure C.1 – Use-case diagram. The only actor is the registered User; the User must log in before using any module except Register and Login.

Annexure D
Data Dictionary (DD)
(Mandatory)

Database: `todo_list` (MySQL 8.0, InnoDB, utf8mb4). Eight tables.

User Table (user_data)

Fields	Data type	Description
username	varchar(50)	Primary key. Unique login name of the user
password	varchar(100)	bcrypt hash of the password (never the plain password)
date_of_birth	date	Date of birth entered at registration
email	varchar(50)	E-mail address; unique key

Todo Task Table (tasks)

Fields	Data type	Description
id	int	Primary key, auto-increment
task	varchar(255)	Task title
status	enum('pending','completed')	Task status; default 'pending'
username	varchar(255)	Owner of the task (links to user_data.username)
deleted_at	timestamp, null	Time of soft delete; NULL while the task is active
is_recurring	tinyint(1)	1 if the task repeats; default 0
recurring_type	enum('daily','weekly','monthly','custom')	Repeat pattern; NULL for normal tasks
next_due	datetime, null	Next date and time at which a recurring task is reset to pending
custom_days	int, null	Interval in days for custom recurrence (1–365)
recurring_time	time, null	Time of day at which a recurring task becomes due
recurring_day_of_week	tinyint, null	Weekday for weekly recurrence (0 = Sunday … 6 = Saturday)
recurring_day_of_month	tinyint, null	Date (1–28) for monthly recurrence
is_pinned	tinyint(1)	1 if the task is pinned to the top; default 0
priority	enum('low','medium','high','critical')	Task priority; default 'medium'
sort_order	int	Position set by drag-and-drop; default 0
due_date	datetime, null	Optional deadline
created_at	timestamp	Creation time; default current time
completed_at	datetime, null	Completion time (reserved for the future statistics feature)

Subtask Table (subtasks)

Fields	Data type	Description
id	int	Primary key, auto-increment
task_id	int	Foreign key to tasks.id, ON DELETE CASCADE; indexed
username	varchar(255)	Owner of the subtask
subtask	varchar(255)	Subtask text
is_done	tinyint(1)	1 when ticked; default 0
created_at	timestamp	Creation time; default current time

Eisenhower Task Table (eisenhower)

Fields	Data type	Description
id	int	Primary key, auto-increment
task	varchar(255)	Task title
quadrant	enum('imp_urg','imp_nurg','nimp_urg','nimp_nurg')	Quadrant: important/urgent, important/not urgent, not important/urgent, not important/not urgent
status	enum('pending','completed')	Task status; default 'pending'
created_at	timestamp	Creation time; default current time
username	varchar(255)	Owner of the task
deleted_at	timestamp, null	Time of soft delete; NULL while active

Thought Pit Table (thought_pits)

Fields	Data type	Description
id	int	Primary key, auto-increment
username	varchar(255)	Owner of the pit; indexed
name	varchar(100)	Name of the thought pit
accent_color	varchar(7)	Hex colour such as #00bc00; default '#00bc00'
created_at	datetime	Creation time; default current time

Node Table (pit_nodes)

Fields	Data type	Description
id	int	Primary key, auto-increment
pit_id	int	Foreign key to thought_pits.id, ON DELETE CASCADE
heading	varchar(120)	Node heading

Fields	Data type	Description
content	text, null	Optional details (up to 1000 characters accepted by the API)
color	varchar(7)	Border colour in hex; default '#8a8a8a'
pos_x	int	Horizontal position on the canvas; default 0
pos_y	int	Vertical position on the canvas; default 0
created_at	datetime	Creation time; default current time

Connection Table (pit_connections)

Fields	Data type	Description
id	int	Primary key, auto-increment
pit_id	int	Foreign key to thought_pits.id, ON DELETE CASCADE
from_node_id	int	Foreign key to pit_nodes.id (start of the arrow), ON DELETE CASCADE
from_side	enum('top','right','bottom','left')	Side of the start node
to_node_id	int	Foreign key to pit_nodes.id (end of the arrow), ON DELETE CASCADE
to_side	enum('top','right','bottom','left')	Side of the end node
created_at	datetime	Creation time; default current time
(unique key)	uniq_link	Combination from_node_id + from_side + to_node_id + to_side must be unique

Quick Notes Table (quick_notes)

Fields	Data type	Description
id	int	Primary key, auto-increment
username	varchar(255)	Owner of the note; indexed together with updated_at
title	varchar(150)	Note title; default ''
preview	varchar(200)	Short text preview shown in the notes list
content	mediumtext, null	Note body stored as HTML (images are embedded)
created_at	datetime	Creation time; default current time
updated_at	datetime	Last change; updated automatically on every save

Annexure E
Screen Shots

Screens are shown in the order in which a user meets them.

Registration page:

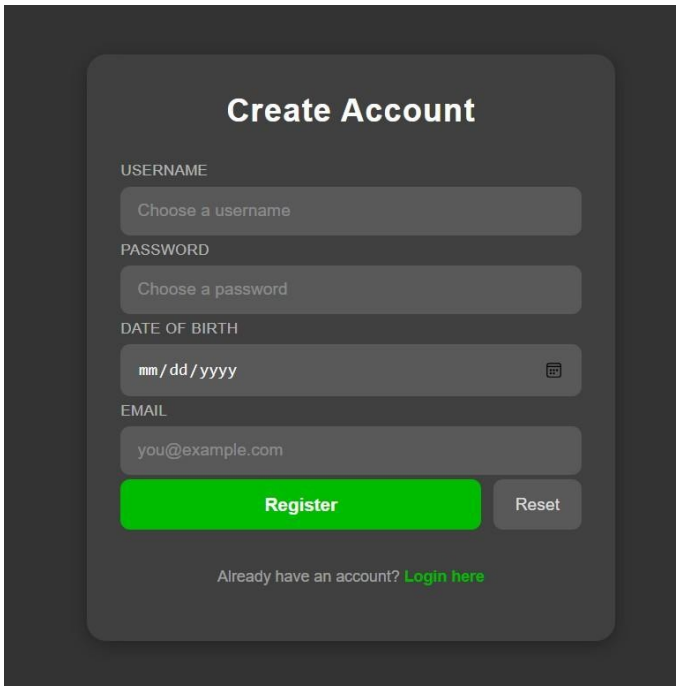


Figure E.1 – Registration page

Login page:

Screenshot to be inserted: Login page

Figure E.2 – Login page

Todo List page (empty list):

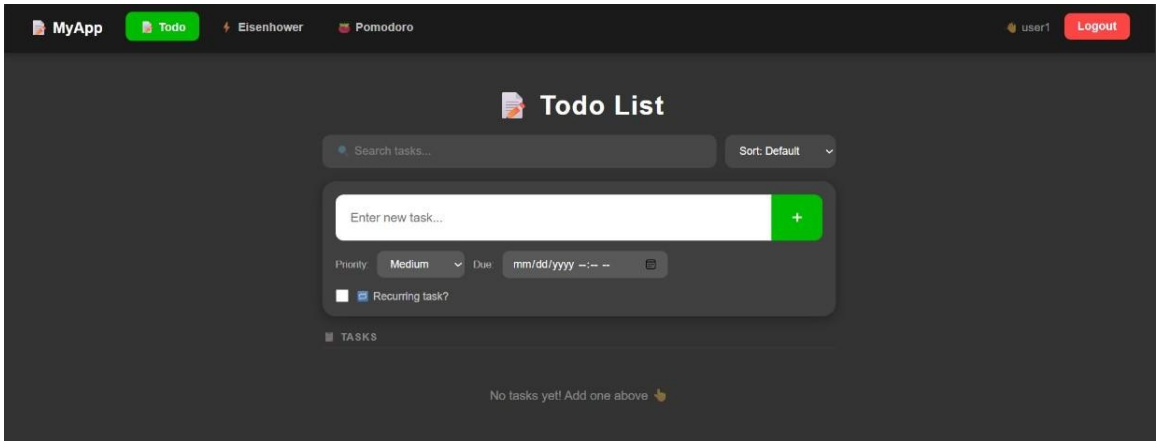


Figure E.3 – Todo List page (empty list)

Todo List – expanded task with subtasks and progress bar:

Screenshot to be inserted: Todo List – expanded task with subtasks and progress bar

Figure E.4 – Todo List – expanded task with subtasks and progress bar

Todo List – Trash section:

Screenshot to be inserted: Todo List – Trash section

Figure E.5 – Todo List – Trash section

Eisenhower Matrix:

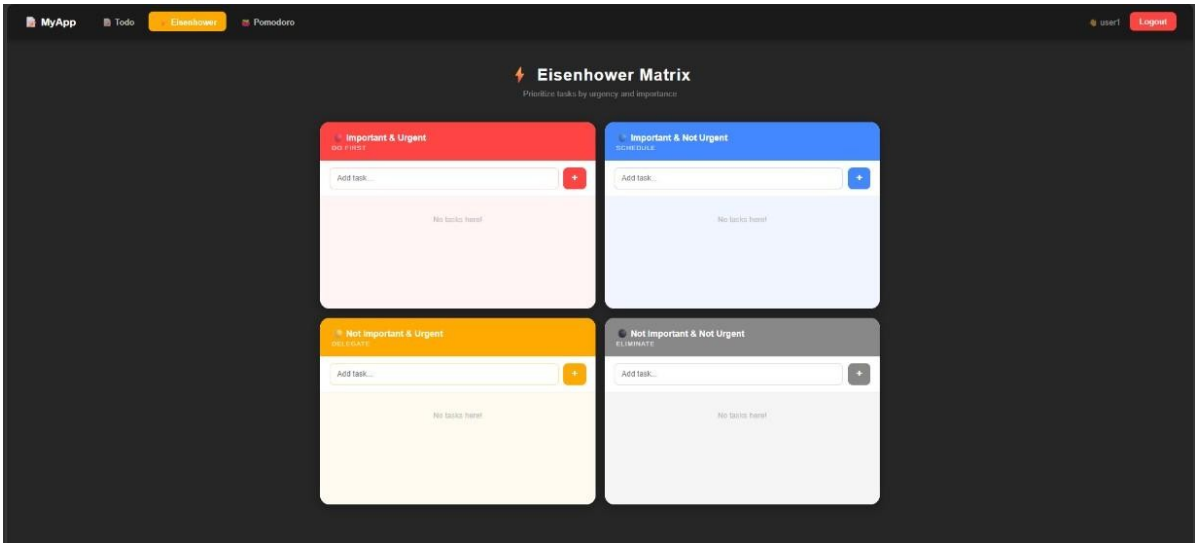


Figure E.6 – Eisenhower Matrix

Pomodoro Timer:

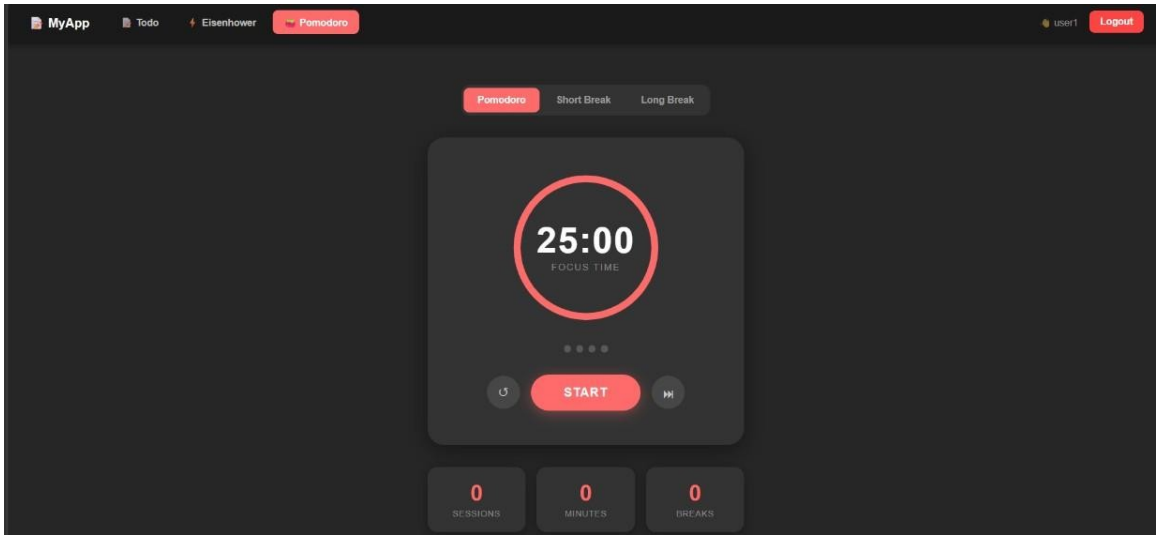


Figure E.7 – Pomodoro Timer

Brain Dump – list of thought pits:

Screenshot to be inserted: Brain Dump – list of thought pits

Figure E.8 – Brain Dump – list of thought pits

Brain Dump – canvas with linked nodes:

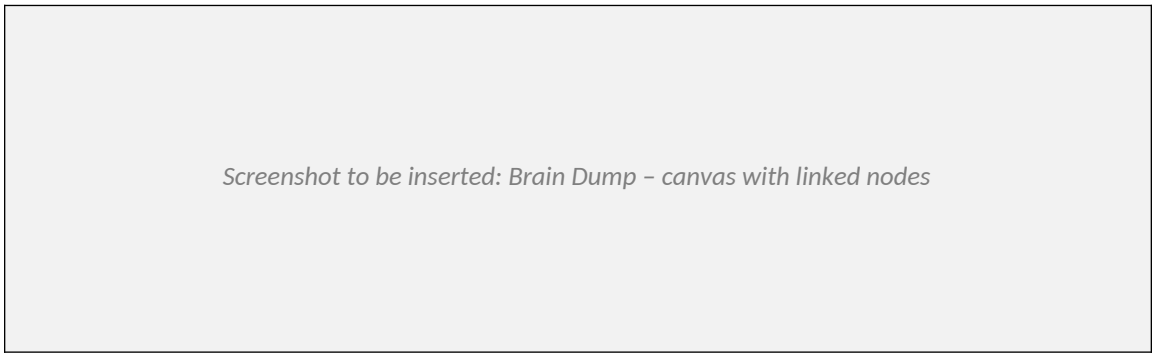


Figure E.9 – Brain Dump – canvas with linked nodes

Quick Notes – rich-text editor:

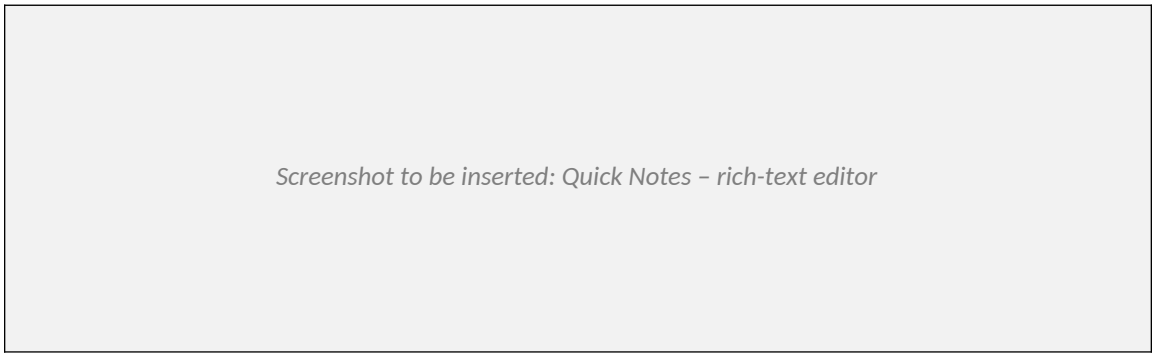


Figure E.10 – Quick Notes – rich-text editor